

Database Schema Reference

Auto-generated schema documentation for LevelUp AI QA Agent.
Last updated: 2026-05-25

Overview

The application uses **PostgreSQL** with raw SQL (no ORM). All tables are created via `CREATE TABLE IF NOT EXISTS` in `src/db/postgres.ts → initSchema()`.

Schema initialization runs **on every server start** — it is fully idempotent.

Health Check

GET `/api/health/database` (no auth required)

Returns the status of all 36 required tables with `healthy: true/false`.

Tables by Domain

Core / Multi-Tenant

Table	Purpose
<code>companies</code>	Multi-tenant company entities
<code>users</code>	User accounts with password hashes, roles
<code>roles</code>	RBAC roles (Owner, QA Manager, QA Engineer, Viewer)
<code>sessions</code>	JWT session tokens
<code>audit_logs</code>	User action audit trail

Test Execution & Self-Healing

Table	Purpose
test_executions	Individual test run records
healing_actions	Healing attempts (rule/pattern/AI) per execution
learned_patterns	Successful healing patterns for reuse
healing_jobs	Async healing job queue (pending → running → completed)
pr_automations	GitHub PRs created from healing fixes

RCA (Root Cause Analysis)

Table	Purpose
rca_analyses	AI-generated root cause analyses for failures

Script Generation (Script Gen)

Table	Purpose
project_contexts	Project settings for Script Gen (app URL, framework, auth, selectors, credentials)
generated_scripts	Generated test scripts (linked to project_contexts)
dom_snapshots	DOM HTML snapshots for script generation
selector_scores	Selector reliability scores
workflow_maps	Page-to-page navigation workflows
generated_projects	Full generated project structures

Test Coverage Intelligence (Test Case Lab)

Table	Purpose
test_requirements	Input requirements / user stories
generated_test_scenarios	AI-generated test scenarios
generated_test_cases	Detailed test cases with steps

Knowledge Management

Table	Purpose
application_knowledge	Module-level app knowledge (workflows, rules, bugs)
knowledge_items	Enterprise knowledge graph nodes
knowledge_relationships	Edges between knowledge items

Repository Intelligence

Table	Purpose
repository_contexts	Scanned repo profiles (framework, patterns)
code_chunks	Code snippets from repo scans

AI Usage & Cost Tracking

Table	Purpose
token_usage	Legacy per-day token usage
ai_usage_logs	Detailed AI call logs (model, tokens, cost, feature)

Notifications

Table	Purpose
notification_configs	Slack/Jira/email notification settings
notification_logs	Notification delivery logs

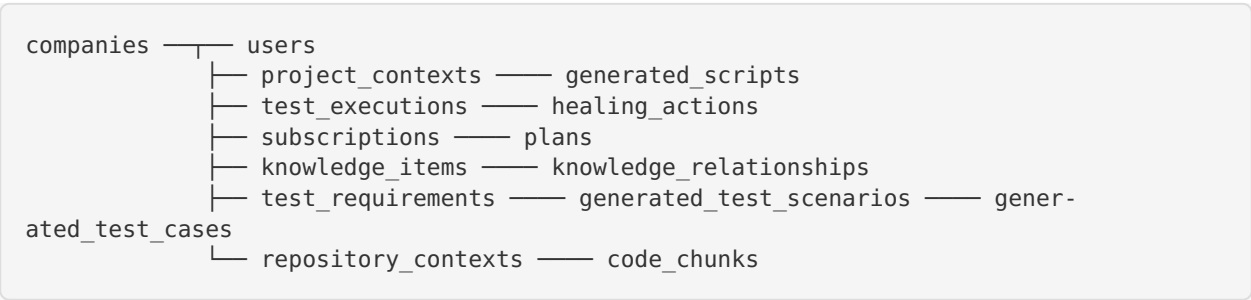
Billing & Licensing

Table	Purpose
plans	Subscription plans (Free, Starter, Growth, Enterprise)
subscriptions	Active company subscriptions
subscription_usage	Credit usage per billing period
billing_events	Payment/invoice events
payment_methods	Stored payment methods (cards)

API & Ingestion

Table	Purpose
api_keys	Machine-to-machine API keys
ingestion_logs	CI/CD test result ingestion records

Key Relationships



Initialization Flow

```
startAPIServer()
├── initDb()
│   ├── initSchema(client) - CREATE TABLE IF NOT EXISTS x 36
│   │   ├── Core tables (test_executions, healing_actions, etc.)
│   │   ├── Script Gen tables (project_contexts, generated_scripts, etc.)
│   │   ├── Billing tables (plans, subscriptions, etc.)
│   │   ├── Knowledge tables (knowledge_items, etc.)
│   │   └── Repository Intelligence tables
│   ├── seedDefaultPlans(client) - Upsert 4 default plans
│   ├── seedDefaultRoles(client) - Upsert 4 default roles
│   ├── migrateDefaultCompany() - Ensure "Default" company + backfill
│   │   └── Migrations[] - ALTER TABLE ADD COLUMN IF NOT EXISTS
│   └── verifySchema(client) - Check all 36 tables exist, log warnings
```

Troubleshooting

Error 42P01 — “relation does not exist”

This means a table hasn't been created yet. Common causes:

1. **Server hasn't been restarted** after a schema update
2. `initDb()` **failed silently** — check startup logs for errors
3. **Migration not applied** — the `DO $$ ... END $$` blocks may have failed

Fix: Restart the server. `initSchema()` uses `CREATE TABLE IF NOT EXISTS` and is safe to re-run. Check `GET /api/health/database` to verify.

Error 23505 — “unique_violation”

A row with duplicate unique key already exists. Check the constraint name in the error's `detail` field.

Error 23503 — “foreign_key_violation”

A referenced row doesn't exist. Common with `company_id` references — ensure the company exists before inserting child records.